COMPLETE BEGINNER-FRIENDLY PROJECT HANDBOOK

# AI-Powered Test Case Generator

A detailed explanation of the complete React, Node.js, Express, Claude AI, testing, and QA workflow project.

React + Vite | Express API | Claude AI | Playwright | QA Engineering

---

Prepared for project demonstration, viva preparation, and code understanding. Project mode: AI-assisted when a key is configured; clearly labelled Demo Mode otherwise.

# Contents


1. What this project solves
2. Main concepts: requirement, test case, AI assistance, and Demo Mode
3. Technology stack and folder structure
4. How the complete system works
5. Frontend screens and user navigation
6. Backend, API, validation, and security
7. Test-case data model and browser storage
8. Exports, quality scoring, bug reports, and traceability
9. JSON suite portability
10. Testing, accessibility, and CI
11. Running the project and optional real AI setup
12. Limitations, future improvements, and viva questions


> **How to read this guide:** Start with the first four sections if you are new to software testing. The later sections map each project feature to the responsible files and explain why it was built that way.

# 1. What problem does this project solve?

Software testers must convert a written requirement into clear test cases before they can check whether an application works correctly. Writing those cases manually can take time, and it is easy to forget negative cases, boundary values, unexpected input, or error handling. This project helps a tester produce a structured first draft, then gives the tester a workspace to review, run, edit, track, export, and improve the cases.

The important idea is that this is **AI-assisted testing**, not automatic proof that an application is correct. A human tester still decides whether a test is relevant, provides actual results, marks a case Pass or Fail, and reviews every AI-generated suggestion. The application makes this honest by visibly labelling each suite as either AI-generated or a Demo result.

# 2. Core concepts in simple words

A **requirement** is a sentence describing what software should do. For example: “Users can sign in with an email address and password.” A **test case** is a repeatable check of that requirement. It contains a scenario, preconditions, steps, test data, expected result, priority, type, and execution status.

A positive test checks normal correct behaviour, such as logging in with valid credentials. A negative test checks rejection or error behaviour, such as entering the wrong password. A boundary test checks limits, such as the shortest acceptable password. An edge-case test checks unusual conditions. Input-validation tests check blank, malformed, or missing values. Using all five categories makes a suite more complete.

## 3. Technology stack and project structure

The user interface uses React 18, a JavaScript library for building pages from reusable components. Vite is the development and production build tool; it serves the React page quickly during development and creates optimized files for deployment. Plain CSS gives the project a light SaaS-style visual design without a large UI framework.

The server uses Node.js and Express. The server is the trusted middle layer between the browser and the Claude API. It keeps the API key on the server, validates requests, contacts Claude when AI mode is enabled, and returns only validated test cases to the browser. This prevents the secret key from being exposed in frontend code.

| Folder / file | Purpose |
|---|---|
| `client/src/App.jsx` | Owns the application state, localStorage saving, page switching, and main callbacks. |
| `client/src/components/` | Small React screen components: Dashboard, Generator, Manage, Sidebar, EditModal, BugReportModal, and Traceability. |
| `client/src/utils/` | Pure JavaScript logic that can be tested without a browser: filtering, CSV, exporters, quality, bug reports, traceability, and suite files. |
| `server/src/app.js` | Express routes, CORS, request parsing, and friendly error responses. |
| `server/src/claude.js` | Builds the Claude prompt, calls the API, and extracts JSON safely. |
| `server/src/validate.js` | Validates requirements and normalizes/filters AI output. |
| `server/src/demo.js` | Produces clearly labelled rule-based demonstration cases when no key exists. |
| `e2e/` | Playwright browser, API, and accessibility tests. |
| `.github/workflows/ci.yml` | GitHub Actions pipeline that runs checks after pushes and pull requests. |

## 4. Architecture: what talks to what?

When a user submits a requirement, the frontend first checks that it is not blank, too short, or too long. It sends a `POST /api/generate` request. The backend repeats validation because browser checks alone are not enough for security. If an API key exists and Demo Mode was not selected, the server requests a JSON-only

response from Claude. Otherwise it uses the local demo generator. In either case, the response is normalized before it reaches the screen.

# 5. Frontend screens and navigation

## Dashboard

The Dashboard is the summary screen. It calculates total cases, Pass, Fail, Blocked, and Not Run counts. The execution progress bar shows how many cases have been run, while the pass rate uses only executed cases. Charts show the distribution by priority and test type. This helps a tester quickly see risk, progress, and gaps without reading every row.

The Dashboard also contains the Test Suite Quality card. It gives a score from 0 to 100 and lists coverage, missing test types, duplicate warnings, and useful review notes. The screen explicitly says that the score is a simple heuristic, not a guarantee of quality.

## Generate Test Cases

The Generator page accepts a plain-English requirement. It provides User Login, Shopping Cart, and Registration samples for a fast demonstration. With no configured key, the page warns that Demo Mode uses predefined/rule-based cases and that they are not AI-generated. With an API key, the page offers real Claude generation and still lets the user deliberately select Demo Mode.

## Manage Test Cases

Manage is the main working area. It displays a table of test cases and supports search by ID or scenario, priority filtering, status filtering, expandable details, editing, deletion, and status changes. Its toolbar exports only the currently visible filtered rows. This is useful when, for example, a QA lead needs only high-priority failed cases.

## Traceability

The Traceability page turns the saved requirement into short points named R1, R2, and so on. It then shows a matrix of each requirement point against the linked cases and their Pass, Fail, Blocked, and Not Run totals. A highlighted “No tests” result warns that a requirement has no coverage. The page also lists cases that remain unlinked.

# 6. Application state and localStorage

`App.jsx` is the coordinator. It stores two important values: `testCases`, which is the current array of cases, and `meta`, which records the source, model, original requirement, generation time, and skipped invalid responses. The state is saved under the browser key `tcg:v1`. Because fields such as `requirementId` are optional, older saved cases still load after the new features are added.

Browser localStorage is convenient for a portfolio application because refreshing the page does not erase work. However, it is not a shared database. Each browser profile has its own saved suite, and clearing browser site data removes it. JSON export is therefore useful as a backup and sharing method.

# 7. Server, API, validation, and safe AI use

The backend begins in `server/src/index.js`. It reads environment variables using dotenv, chooses the port, determines whether `ANTHROPIC_API_KEY` is available, creates the Express app, and starts listening. The default development port is 5000, although a different local port can be used when another program already owns 5000.

`GET /api/health` tells the frontend whether AI mode is available. It returns a small object containing `ok`, `aiConfigured`, and the model name when applicable. This is why the sidebar can truthfully show “Demo mode” or “Claude AI mode.”

`POST /api/generate` receives `requirement` and `useDemo`. It rejects invalid input with HTTP 400. For valid input, it chooses demo generation when no key exists or the request explicitly asks for demo mode. If the AI call fails because of a bad key, rate limit, timeout, or invalid output, the server returns a meaningful error. It does not silently replace failed AI output with demo data, because that would falsely claim an AI result.

> **Why the key is safe here:** the browser calls the project server, and only the server reads `ANTHROPIC_API_KEY` from `server/.env`. The frontend source code never contains the key. The `.env` file is ignored by Git and must never be uploaded to GitHub.

### Validation pipeline

Validation has two layers. The client gives an immediate friendly error to the user. The server repeats the check because any outside client could call the endpoint directly. AI output is then checked for the required test-case shape, allowed priority, allowed type, usable steps, and text fields. Invalid items are skipped, valid items get IDs such as TC-001, and each starts as Not Run.

# 8. The test-case data model

| Field | Meaning |
|---|---|
| id | Stable identifier such as TC-001. |
| scenario / description | Short title and fuller explanation of what is tested. |
| preconditions / steps / testData | What must be true first, how to perform the test, and values to use. |
| expectedResult | What should happen if the feature works correctly. |
| priority / type / status | Risk level, QA category, and current execution outcome. |
| requirementId | Optional link to R1, R2, and other requirement points. |

# 9. Exports and automation starters

### CSV export

CSV is a simple spreadsheet-compatible format. The application exports visible rows with columns for the ID, scenario, description, preconditions, numbered steps, test data, expected result, priority, type, status, and linked requirement. Cells are quoted correctly, quotation marks are escaped, and values beginning with spreadsheet formula characters are neutralized. This protects against CSV formula injection when someone opens the file in Excel.

### Playwright export

The Playwright export creates a JavaScript test file. It imports `test` and `expect`, creates one describe block, and produces one named test for each test case. Preconditions, steps, test data, and expected results become readable comments. Every exported test has `test.fixme(true, "Not automated yet")`. This is deliberate: a requirement has no URL, CSS selectors, or real target application actions, so pretending the export is fully automated would be misleading.

### Gherkin / BDD export

The Gherkin export creates a `.feature` file that teams can read easily. It uses a Feature heading, tags like `@TC001 @High @Negative`, and Given/When/Then language. Preconditions become Given, the first step becomes When, later steps become And, and the expected result becomes Then. This format helps developers, testers, and non-technical stakeholders discuss expected behaviour together.

# 10. Test-suite quality score

The quality function in `client/src/utils/quality.js` does not judge whether a case is logically perfect. Instead, it measures four visible completeness signals. It gives 40 points when all five types are represented, 20 points when no near-duplicate scenarios are found, 20 points based on concrete test data, and 20 points based on sufficiently detailed expected results.

Duplicate detection creates simple word sets from scenarios, removes common words, and calculates Jaccard similarity. If two scenarios have similarity of at least 0.8, they are flagged as likely duplicates. “N/A”, blank, or very short data is not treated as concrete. An expected result needs at least 25 characters and five words to count as specific. These rules are intentionally simple and explainable in a viva.

## 11. Bug reports and traceability

When a case is marked Fail, a Bug report button appears. The modal pre-fills information that is already known: test case ID, scenario, priority, preconditions, steps, test data, and expected result. It suggests severity from priority: High becomes Major, Medium becomes Moderate, and Low becomes Minor. The tester must enter the actual result, because the application must not invent evidence about a failure.

The generated Markdown report has a clear title, severity, priority, environment, preconditions, numbered steps to reproduce, test data, expected result, actual result, and linked test-case ID. The user can copy the report or download it as a `.md` file. Markdown is useful because it can be pasted into GitHub Issues, Jira descriptions, or documentation.

Traceability is the link between what a customer requested and the checks that prove it was tested. The project splits requirement text at sentences or new lines and labels the resulting points R1, R2, and so on. A tester can choose a linked requirement while editing a case. The Auto-link button makes a keyword-overlap suggestion only for unlinked cases. It never replaces human judgement.

## 12. JSON suite import and export

JSON export saves the complete portable suite as an object containing a version number, export time, metadata, and test cases. The metadata is important because it preserves whether a suite came from AI or Demo Mode. This avoids a misleading label after sharing a file with another person.

Before importing JSON, `parseSuite` checks that the file is valid JSON, uses version 1, contains a test-case array, includes all required fields, uses allowed priority/type/status values, has non-empty steps, and has no duplicate IDs. Invalid files show a clear error and do not replace the current work. If existing cases are present, the application asks for confirmation before replacing them.

> **Important:** importing a suite preserves the source metadata stored in the file. It does not transform Demo data into AI data or claim that imported cases were generated by Claude.

## 13. Automated testing and accessibility

Unit tests use Node’s built-in test runner. They test pure functions, including validation, statistics, filters, status updates, CSV safety, exporters, quality scoring, bug-report Markdown, traceability logic, and JSON parsing. Pure functions are easier to test because they accept input and return output without requiring a browser.

Playwright E2E tests open the application as a user would. They confirm that Demo Mode appears, validation prevents short requirements, sample cases open correctly, status changes update the Dashboard, filters work together, editing and deletion work, downloads occur, localStorage survives a reload, API responses work, and accessibility checks have no serious or critical violations. Axe is used to detect accessibility problems such as missing labels or insufficient semantic information.

# 14. CI, installation, and running the project

GitHub Actions provides Continuous Integration. Whenever the project is pushed to GitHub or a pull request is opened, the workflow installs Node 20 dependencies, runs the server tests, runs the client tests, installs Chromium for Playwright, runs the browser suite, and uploads the HTML browser-test report as an artifact. CI helps detect a regression before code is merged.

```
# First-time setup
cd server && npm install
cd ../client && npm install
cd ../e2e && npm install
npx playwright install chromium

# Run the application in two terminals
cd server && npm start
cd client && npm run dev

# Run checks from the project root
npm run test:server
npm run test:client
npm run test:e2e
npm run test:all
```

### Optional real Claude AI mode

Real AI is optional. Demo Mode costs nothing and is appropriate for a project viva. To use Claude, create a prepaid Anthropic API account, disable automatic credit reload if you want a strict spending limit, create an API key, and place it only in `server/.env`. After restarting the server, the sidebar reports Claude AI mode and the Generate page can send a real request.

```
# server/.env — never commit this file
ANTHROPIC_API_KEY=your_private_key_here
CLAUDE_MODEL=claude-sonnet-5-5
PORT=5000
```

The API is billed separately from a Claude chat subscription and only successful AI requests use prepaid credits. Never paste the key into the browser, a screenshot, an assignment document, or a public GitHub repository.

# 15. Limitations and sensible future improvements

The system stores data only in one browser, so it has no accounts, shared projects, database history, or team permissions. The demo generator recognizes only a few sample categories, while AI output can still be incomplete, repetitive, or incorrect. The quality score and auto-linking are heuristic suggestions. Exported Playwright tests require a human to add a target URL, page locators, and real assertions.

Useful future work includes database storage, multi-project support, test-run history, PDF/Excel reports, real issue-tracker integration, requirement import from Jira, authentication, rate limiting, configurable quality rules, and an interface for mapping generated tests to a real application under test.

## 16. Short viva answers

**Why is there both client and server validation?** Client validation gives fast feedback, but server validation is needed because a malicious or external caller can bypass the browser.

**Why is Demo Mode labelled so clearly?** It prevents a portfolio demonstration from incorrectly claiming that predefined cases were produced by AI.

**Why keep utility functions separate from React components?** Small pure functions are easier to understand, test, reuse, and explain than logic hidden inside a large UI component.

**Why is the API key on the server?** Anything sent to browser JavaScript can be viewed by users. The server keeps the secret private and controls calls to the AI provider.

**What is the difference between unit and E2E testing?** Unit testing checks a small function in isolation. E2E testing checks a complete user journey through the running application.

**Does a score of 100 mean the suite is perfect?** No. It only means this simple checklist found good coverage and detail. Human review remains essential.

## 17. Automation Lab: real UI automation evidence

The Automation Lab makes this project stronger for an automation QA internship. It includes a small, deterministic Login Application Under Test (AUT) inside the same React interface. A user can enter an email address and password, submit the form, receive validation feedback, or reach a signed-in state. This is different from a static demonstration because it provides controls and visible outcomes for a browser automation framework to test.

Five dedicated Playwright cases automate this AUT: a successful login with the known local credentials, blank-field validation, invalid-email validation, rejected wrong credentials, and the minimum eight-character password boundary. The tests use stable accessible locators and assertions against the real validation message or signed-in screen. This demonstrates positive, negative, input-validation, and boundary testing in an executable form.

The Login AUT deliberately does not claim to be production authentication. A real login system requires a secure server, password hashing, rate limiting, protected sessions, and a database. Its purpose is to demonstrate repeatable automation skills without relying on an unreliable external website or exposing any user account.

> **Final project summary:** This application turns a plain-English requirement into reviewable QA test cases, supports truthful AI or Demo generation, tracks execution, improves suite quality, connects tests to requirements, creates bug reports, exports useful artifacts, validates portable suites, includes a real Login AUT with executable Playwright checks, and verifies itself with unit, browser, API, accessibility, and CI checks.